

Systems-Level Analysis of Parameter-Efficient Fine-Tuning Methods

Keyuan Yan
Rice University
Houston, TX, USA
ky65@rice.edu

Abstract

Parameter-Efficient Fine-Tuning (PEFT) methods are evaluated in the literature primarily on trainable-parameter count and task accuracy, yet deployment decisions—which GPU, which batch size, whether a model fits at all—depend on *peak GPU memory*, training throughput, and interactions with system optimizations. We benchmark six PEFT methods (LoRA, QLoRA, DoRA, QDoRA, AdaLoRA, and full fine-tuning) on Microsoft Phi-2 (2.7B) with the Alpaca dataset, using identical hardware (NVIDIA L40S) for all 20 experiments across four phases: method comparison, rank sensitivity, gradient-checkpointing interactions, and kernel-level profiling. We find that DoRA uses 47 % more peak memory than LoRA and 24 % more than full fine-tuning—an overhead the DoRA paper acknowledges but the default HuggingFace PEFT implementation does not mitigate. We further show that gradient checkpointing reduces PEFT memory by 68–77 % but provides *zero* savings for full fine-tuning, revealing a complementarity: PEFT eliminates optimizer-state memory while gradient checkpointing eliminates activation memory. Together they enable QLoRA + GC to fine-tune a 2.7B-parameter model in 5.0 GB.

Keywords

LoRA, QLoRA, DoRA, Gradient Checkpointing, Memory Profiling, PEFT

1 Introduction

Large language models are routinely adapted to downstream tasks via fine-tuning. Full fine-tuning updates every parameter and is prohibitively expensive for billion-parameter models because AdamW stores two additional state tensors per parameter. PEFT methods sidestep this by freezing the base weights and training a small number of additional parameters. LoRA [1] trains a low-rank decomposition $B \cdot A$ of the weight delta. QLoRA [2] further quantizes the frozen weights to 4-bit NF4. DoRA [3] decomposes pretrained weights into magnitude and direction and applies LoRA only to the direction. AdaLoRA [4] prunes ranks dynamically.

These papers report trainable-parameter counts and task accuracy. For practitioners, a different set of questions is operationally urgent. (RQ1) How much GPU memory does each method actually consume? (RQ2) How does throughput differ? (RQ3) How does rank affect these systems metrics? (RQ4) How do PEFT methods interact with gradient checkpointing?

Measuring all four on identical hardware surfaces two non-obvious findings. First, DoRA uses *more* peak memory than full fine-tuning, contradicting the intuition that PEFT methods always save memory. Second, gradient checkpointing saves 68–77 % for

Table 1: Experiment matrix (20 runs, 4 phases).

Phase	Experiments	Purpose
1	full_ft, lora_r16, qlora_r16, dora_r16, qdora_r16, adalora_r16	Method comparison
2	lora_r{4,8,16,32,64,128}	Rank sensitivity
3	lora/dora/qlora/full_ft_gc	GC interactions
4	Profile of full_ft, lora, qlora, dora	Kernel breakdown

PEFT methods but *zero* for full fine-tuning—a pattern that illuminates where PEFT’s memory savings actually come from.

2 Background

LoRA replaces the weight update ΔW with a low-rank factorization BA , where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $r \ll \min(d, k)$. Only B and A are trained; the frozen W remains in memory.

QLoRA reduces frozen-weight memory by storing W in 4-bit NormalFloat and dequantizing on the fly during the forward pass.

DoRA decomposes $W = m \cdot (V/\|V\|)$, where m is a per-column magnitude and V is the direction matrix; it trains m directly and adapts V via LoRA, so $V' = V + BA$ and the effective weight is $W' = m' \cdot (V'/\|V'\|)$. The normalization $\|V'\|$ requires materializing the full V' matrix during backpropagation. The DoRA paper explicitly notes this overhead and proposes detaching $\|V'\|$ from the gradient graph, reporting a 24.4 % memory reduction on LLaMA-7B [3].

3 Experimental Setup

All 20 experiments ran on a single NVIDIA L40S (47.6 GB, Ada Lovelace) via SLURM on the Rice NOTS cluster. The base model is Microsoft Phi-2 (2.7B parameters, MIT license) and the dataset is Alpaca (52K instruction-following samples). Each run trains for 500 steps with sequence length 512, effective batch 16 (per-device batch $4 \times$ gradient accumulation 4), bf16 precision, and AdamW. Software stack: PyTorch 2.5.1, HuggingFace transformers 4.44, PEFT 0.12, bitsandbytes 0.43. Flash Attention was unavailable on the cluster; *all* methods used the same PyTorch SDPA kernel, so relative comparisons are fair.

For each experiment we record peak GPU memory via `torch.cuda.max_memory_allocated()`, per-step wall time with CUDA synchronization, training loss, and trainable-parameter count. For Phase 4 we run `torch.profiler` with CUDA activity tracing over 11 training steps per method (1 wait, 2 warmup, 8 active) on four canonical methods.

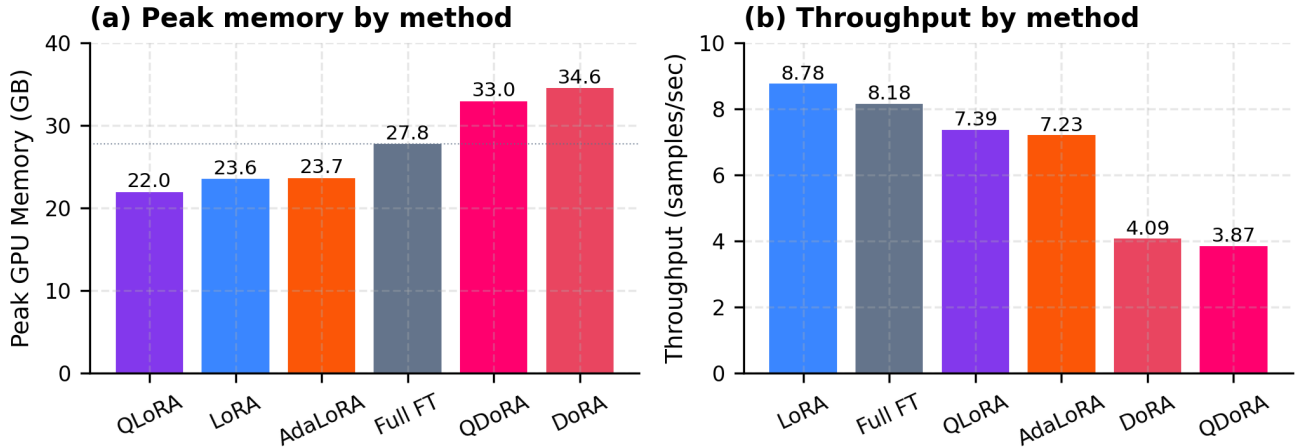


Figure 1: Peak memory (left) and throughput (right) across six methods. DoRA and QDoRA use substantially more memory than full fine-tuning despite having $\sim 1\%$ of the trainable parameters; LoRA achieves the highest throughput.

4 Results

4.1 Method comparison (RQ1, RQ2)

Table 2 summarizes Phase 1. LoRA is the fastest method (8.78 samples/s) and achieves the lowest training loss (0.159). DoRA, despite only 0.03% more trainable parameters than LoRA, uses 11.0 GB more peak memory (+47%) and runs 2.15 $\times$ slower. QDoRA mirrors this pattern. AdaLoRA’s rank pruning prevents convergence at 500 steps.

Table 2: Phase 1 results ($r=16$ for all PEFT methods).

Method	Mem (GB)	Throughput	Loss	Train%
Full FT	27.8	8.18 s/s	0.177	100.00
LoRA	23.6	8.78 s/s	0.159	0.84
QLoRA	22.0	7.39 s/s	0.161	1.53
DoRA	34.6	4.09 s/s	0.164	0.87
QDoRA	33.0	3.87 s/s	0.167	1.57
AdaLoRA	23.7	7.23 s/s	0.298	0.84

4.2 Rank sensitivity (RQ3)

Sweeping LoRA rank from 4 to 128 (a 32 $\times$ increase) adds only 3.4 GB of peak memory and reduces throughput by 3% (Fig. 2). The frozen Phi-2 weights and activations—approximately 21 GB—dominate memory, making LoRA matrices proportionally small even at $r = 128$. Training loss shows no consistent improvement beyond $r = 16$ (0.159 at $r = 16$ vs. 0.158 at $r = 64$), confirming Hu et al.’s observation [1] that rank is relatively insensitive for accuracy. We add the complementary *systems* observation that rank is also relatively insensitive for memory and throughput.

4.3 Gradient checkpointing (RQ4)

Gradient checkpointing recomputes activations during the backward pass instead of storing them. For PEFT methods, peak memory is dominated by activations (optimizer states are tiny because only ~ 23 M parameters are trained), so GC eliminates most of it: LoRA drops 23.6 $\rightarrow$ 7.6 GB (−68%), QLoRA 22.0 $\rightarrow$ 5.0 GB (−77%), and

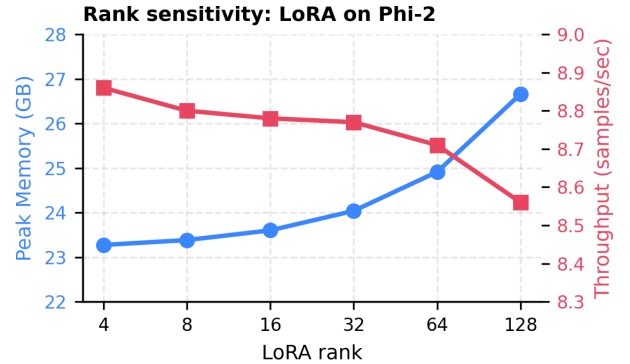


Figure 2: LoRA rank 4 $\rightarrow$ 128. Memory grows only 3.4 GB; throughput drops only 3%.

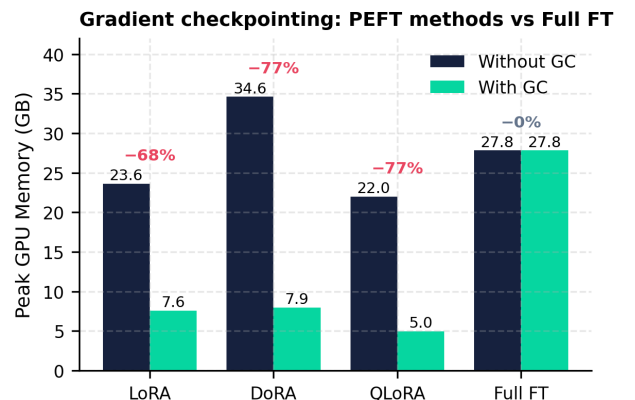


Figure 3: Gradient checkpointing reduces PEFT peak memory by 68–77% but yields 0% reduction for full fine-tuning.

DoRA 34.6 $\rightarrow$ 7.9 GB (−77%). Full fine-tuning, by contrast, sees zero memory reduction: its memory is dominated by AdamW optimizer

states, which at $2.78\text{B} \times 2 \times 4 \text{ bytes} \approx 22 \text{ GB}$ in fp32 cannot be checkpointed. The throughput cost of GC ranges from 22 % (Full FT) to 34 % (DoRA).

4.4 Kernel profiling

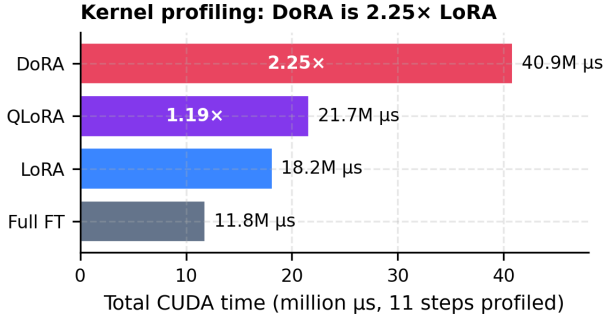


Figure 4: Total CUDA time per method (11 profiled training steps).

The profiler confirms the throughput measurements (Fig. 4): DoRA requires $2.25\times$ LoRA’s total CUDA time, QLoRA requires $1.19\times$. Keyword-based kernel categorization is inconclusive for a finer breakdown—DoRA’s weight normalization decomposes into primitive operations (matrix multiply, element-wise multiply, dtype cast) that land in an “Other” bucket, and bitsandbytes fuses dequantization into its GEMM kernels. We therefore use total CUDA time as the primary signal.

5 Discussion

5.1 Does our DoRA result contradict the paper?

No. The DoRA paper explicitly discusses this overhead and proposes detaching $\|V'\|$ from the gradient graph as a mitigation, reporting a 24.4 % memory reduction on LLaMA-7B [3]. Our 47 % overhead on Phi-2 is consistent with running without that optimization. Notably, answer.ai’s production FSDP + QDoRA pipeline [5] claims DoRA is “just as memory efficient” as QLoRA—but their training scripts enable gradient checkpointing by default. Our results confirm this: with GC, DoRA (7.9 GB) $\approx$ LoRA (7.6 GB). The practical takeaway is that DoRA deployed via HuggingFace PEFT effectively *requires* gradient checkpointing, a dependency rarely made explicit to practitioners.

5.2 PEFT and GC are complementary

PEFT and gradient checkpointing attack different sources of memory. Full-parameter training spends most memory on optimizer states; PEFT eliminates this by freezing most parameters. Activation memory is then the bottleneck, which GC attacks by recomputation. Neither alone achieves QLoRA + GC’s 5.0 GB floor: LoRA alone needs 23.6 GB; Full FT + GC needs 27.8 GB. The combination enables a 2.7B-parameter model to fine-tune on a commodity 8 GB GPU.

5.3 Limitations

(i) **Flash Attention** was not installed on our cluster; all methods used SDPA. This inflates absolute memory by roughly 2 GB uniformly but does not affect relative comparisons, because the DoRA–LoRA gap originates from weight normalization, not attention. (ii) **We benchmark one model scale** (2.7B); the normalization overhead in DoRA scales with layer dimensions, so larger models may show different proportional overhead. (iii) **Kernel categorization** via keyword matching is imperfect; we rely on total CUDA time as the primary signal. (iv) **AdaLoRA’s high loss** (0.298) reflects an under-trained rank pruning schedule rather than a fundamental flaw of the method.

6 Conclusion

A systems-level benchmark of six PEFT methods on identical hardware reveals two non-obvious findings. DoRA, as implemented in HuggingFace PEFT, uses 47 % more peak memory than LoRA and 24 % more than full fine-tuning—an overhead the DoRA paper acknowledges but the default implementation does not mitigate. Gradient checkpointing reduces PEFT memory by 68–77 % but produces no savings for full fine-tuning, demonstrating that PEFT and GC attack complementary memory sources (optimizer state vs. activations). The practical recommendation is to enable gradient checkpointing whenever deploying PEFT methods—especially DoRA—even when published numbers suggest the bare method is already memory-efficient.

Reproducibility

All code, SLURM scripts, and raw results are included in the submission. Each experiment is a single command: `python src/train.py --experiment <name>`. Figures regenerate with `python analysis/plot_results.py`.

References

- [1] E. J. Hu, Y. Shen, P. Wallis, et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
- [2] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS, 2023.
- [3] S.-Y. Liu, C.-Y. Wang, H. Yin, et al. *DoRA: Weight-Decomposed Low-Rank Adaptation*. ICML, 2024. arXiv:2402.09353.
- [4] Q. Zhang, M. Chen, A. Bukharin, et al. *AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning*. ICLR, 2023.
- [5] Answer.AI. *Fine-tune Llama 3 with FSDP-QDoRA*. <https://www.answer.ai/posts/2024-04-26-fsdp-qdora-llama3.html>, April 2024.
- [6] HuggingFace. *PEFT Library*. <https://github.com/huggingface/peft>.
- [7] Microsoft. *Phi-2 (2.7B)*. <https://huggingface.co/microsoft/phi-2>.
- [8] R. Taori, I. Gulrajani, T. Zhang, et al. *Stanford Alpaca: An Instruction-Following LLaMA Model*. 2023.